JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time

Class YearMonth

java.lang.Object
    java.time.YearMonth

All Implemented Interfaces:

`Serializable`, `Comparable<YearMonth>`, `Temporal`, `TemporalAccessor`, `TemporalAdjuster`

```
public final class YearMonth
extends Object
implements Temporal, TemporalAdjuster, Comparable<YearMonth>, Serializable
```

A year-month in the ISO-8601 calendar system, such as `2007-12`.

`YearMonth` is an immutable date-time object that represents the combination of a year and month. Any field that can be derived from a year and month, such as quarter-of-year, can be obtained.

This class does not store or represent a day, time or time-zone. For example, the value "October 2007" can be stored in a `YearMonth`.

The ISO-8601 calendar system is the modern civil calendar system used today in most of the world. It is equivalent to the proleptic Gregorian calendar system, in which today's rules for leap years are applied for all time. For most applications written today, the ISO-8601 rules are entirely suitable. However, any application that makes use of historical dates, and requires them to be accurate will find the ISO-8601 approach unsuitable.

This is a value-based class; use of identity-sensitive operations (including reference equality (==), identity hash code, or synchronization) on instances of `YearMonth` may have unpredictable results and should be avoided. The `equals` method should be used for comparisons.

Implementation Requirements:

This class is immutable and thread-safe.

Since:

1.8

See Also:

Serialized Form

Method Summary

Concrete Methods **Instance Methods**

Modifier and Type	Method	Description
Temporal	adjustInto(Temporal temporal)	Adjusts the specified temporal object to have this year-month.
LocalDate	atDay(int dayOfMonth)	Combines this year-month with a day-of-month to create a LocalDate.
LocalDate	atEndOfMonth()	Returns a LocalDate at the end of the month.
int	compareTo(YearMonth other)	Compares this year-month to another year-month.
boolean	equals(Object obj)	Checks if this year-month is equal to another

		year-month.
String	format(DateTimeFormatter formatter)	Formats this year-month using the specified formatter.
static YearMonth	from(TemporalAccessor temporal)	Obtains an instance of YearMonth from a temporal object.
int	get(TemporalField field)	Gets the value of the specified field from this year-month as an int.
long	getLong(TemporalField field)	Gets the value of the specified field from this year-month as a long.
Month	getMonth()	Gets the month-of-year field using the Month enum.
int	getMonthValue()	Gets the month-of-year field from 1 to 12.
int	getYear()	

		Gets the year field.
int	**hashCode**()	A hash code for this year-month.
boolean	**isAfter**(**YearMonth** other)	Checks if this year-month is after the specified year-month.
boolean	**isBefore**(**YearMonth** other)	Checks if this year-month is before the specified year-month.
boolean	**isLeapYear**()	Checks if the year is a leap year, according to the ISO proleptic calendar system rules.
boolean	**isSupported**(**TemporalField** field)	Checks if the specified field is supported.
boolean	**isSupported**(**TemporalUnit** unit)	Checks if the specified unit is supported.
boolean	**isValidDay**(int dayOfMonth)	Checks if the day-of-month is

		valid for this year-month.
int	**lengthOfMonth**()	Returns the length of the month, taking account of the year.
int	**lengthOfYear**()	Returns the length of the year.
YearMonth	**minus**(long amountToSubtract, **TemporalUnit** unit)	Returns a copy of this year-month with the specified amount subtracted.
YearMonth	**minus**(**TemporalAmount** amountToSubtract)	Returns a copy of this year-month with the specified amount subtracted.
YearMonth	**minusMonths**(long monthsToSubtract)	Returns a copy of this YearMonth with the specified number of months subtracted.
YearMonth	**minusYears**(long yearsToSubtract)	Returns a copy of this YearMonth with the specified

		number of years subtracted.
static YearMonth	now()	Obtains the current year-month from the system clock in the default time-zone.
static YearMonth	now(Clock clock)	Obtains the current year-month from the specified clock.
static YearMonth	now(ZoneId zone)	Obtains the current year-month from the system clock in the specified time-zone.
static YearMonth	of(int year, int month)	Obtains an instance of YearMonth from a year and month.
static YearMonth	of(int year, Month month)	Obtains an instance of YearMonth from a year and month.
static YearMonth	parse(CharSequence text)	

		Obtains an instance of YearMonth from a text string such as 2007-12.
static YearMonth	parse(CharSequence text, DateTimeFormatter formatter)	Obtains an instance of YearMonth from a text string using a specific formatter.
YearMonth	plus(long amountToAdd, TemporalUnit unit)	Returns a copy of this year-month with the specified amount added.
YearMonth	plus(TemporalAmount amountToAdd)	Returns a copy of this year-month with the specified amount added.
YearMonth	plusMonths(long monthsToAdd)	Returns a copy of this YearMonth with the specified number of months added.
YearMonth	plusYears(long yearsToAdd)	Returns a copy of

		this YearMonth with the specified number of years added.
<R> R	**query**(**TemporalQuery**<R> query)	Queries this year-month using the specified query.
ValueRange	**range**(**TemporalField** field)	Gets the range of valid values for the specified field.
String	**toString**()	Outputs this year-month as a String, such as 2007-12.
long	**until**(**Temporal** endExclusive, **TemporalUnit** unit)	Calculates the amount of time until another year-month in terms of the specified unit.
YearMonth	**with**(**TemporalAdjuster** adjuster)	Returns an adjusted copy of this year-month.
YearMonth		

	with(**TemporalField** field, long newValue)	Returns a copy of this year-month with the specified field set to a new value.
YearMonth	**withMonth**(int month)	Returns a copy of this YearMonth with the month-of-year altered.
YearMonth	**withYear**(int year)	Returns a copy of this YearMonth with the year altered.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Method Detail

now

```
public static YearMonth now()
```

Obtains the current year-month from the system clock in the default time-zone.

This will query the system clock in the default time-zone to obtain the current year-month.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Returns:

the current year-month using the system clock and default time-zone, not null

now

```
public static YearMonth now(ZoneId zone)
```

Obtains the current year-month from the system clock in the specified time-zone.

This will query the `system clock` to obtain the current year-month. Specifying the time-zone avoids dependence on the default time-zone.

Using this method will prevent the ability to use an alternate clock for testing because the clock is hard-coded.

Parameters:

zone - the zone ID to use, not null

Returns:

the current year-month using the system clock, not null

now

```
public static YearMonth now(Clock clock)
```

Obtains the current year-month from the specified clock.

This will query the specified clock to obtain the current year-month. Using this method allows the use of an alternate clock for testing. The alternate clock may be introduced using `dependency injection`.

Parameters:

clock - the clock to use, not null

Returns:

the current year-month, not null

of

```
public static YearMonth of(int year,
                           Month month)
```

Obtains an instance of YearMonth from a year and month.

Parameters:

year - the year to represent, from MIN_YEAR to MAX_YEAR

month - the month-of-year to represent, not null

Returns:

the year-month, not null

Throws:

DateTimeException - if the year value is invalid

of

```
public static YearMonth of(int year,
                           int month)
```

Obtains an instance of YearMonth from a year and month.

Parameters:

year - the year to represent, from MIN_YEAR to MAX_YEAR

month - the month-of-year to represent, from 1 (January) to 12 (December)

Returns:

the year-month, not null

Throws:

DateTimeException - if either field value is invalid

from

```
public static YearMonth from(TemporalAccessor temporal)
```

Obtains an instance of YearMonth from a temporal object.

This obtains a year-month based on the specified temporal. A TemporalAccessor represents an arbitrary set of date and time information, which this factory converts to an instance of YearMonth.

The conversion extracts the YEAR and MONTH_OF_YEAR fields. The extraction is only permitted if the temporal object has an ISO chronology, or can be converted to a LocalDate.

This method matches the signature of the functional interface TemporalQuery allowing it to be used as a query via method reference, YearMonth::from.

Parameters:

temporal - the temporal object to convert, not null

Returns:

the year-month, not null

Throws:

`DateTimeException` - if unable to convert to a YearMonth

parse

```
public static YearMonth parse(CharSequence text)
```

Obtains an instance of `YearMonth` from a text string such as `2007-12`.

The string must represent a valid year-month. The format must be `uuuu-MM`. Years outside the range 0000 to 9999 must be prefixed by the plus or minus symbol.

Parameters:

`text` - the text to parse such as "2007-12", not null

Returns:

the parsed year-month, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

parse

```
public static YearMonth parse(CharSequence text,
                              DateTimeFormatter formatter)
```

Obtains an instance of `YearMonth` from a text string using a specific formatter.

The text is parsed using the formatter, returning a year-month.

Parameters:

`text` - the text to parse, not null

`formatter` - the formatter to use, not null

Returns:

the parsed year-month, not null

Throws:

`DateTimeParseException` - if the text cannot be parsed

isSupported

```
public boolean isSupported(TemporalField field)
```

Checks if the specified field is supported.

This checks if this year-month can be queried for the specified field. If false, then calling the `range`, `get` and `with(TemporalField, long)` methods will throw an exception.

If the field is a `ChronoField` then the query is implemented here. The supported fields are:

- `MONTH_OF_YEAR`
- `PROLEPTIC_MONTH`
- `YEAR_OF_ERA`
- `YEAR`
- `ERA`

All other `ChronoField` instances will return false.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.isSupportedBy(TemporalAccessor)` passing `this` as the argument. Whether the field is supported is determined by the field.

Specified by:

`isSupported` in interface `TemporalAccessor`

Parameters:

`field - the field to check, null returns false`

Returns:

`true if the field is supported on this year-month, false if not`

isSupported

`public boolean isSupported(TemporalUnit unit)`

Checks if the specified unit is supported.

This checks if the specified unit can be added to, or subtracted from, this year-month. If false, then calling the `plus(long, TemporalUnit)` and `minus` methods will throw an exception.

If the unit is a `ChronoUnit` then the query is implemented here. The supported units are:

- `MONTHS`
- `YEARS`
- `DECADES`
- `CENTURIES`
- `MILLENNIA`
- `ERAS`

All other `ChronoUnit` instances will return false.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.isSupportedBy(Temporal)` passing `this` as the argument. Whether the unit is supported is determined by the unit.

Specified by:

`isSupported` in interface `Temporal`

Parameters:

`unit - the unit to check, null returns false`

Returns:

`true if the unit can be added/subtracted, false if not`

range

`public ValueRange range(TemporalField field)`

Gets the range of valid values for the specified field.

The range object expresses the minimum and maximum valid values for a field. This year-month is used to enhance the accuracy of the returned range. If it is not possible to return the range, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the query is implemented here. The `supported fields` will return appropriate range instances. All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.rangeRefinedBy(TemporalAccessor)` passing `this` as the argument. Whether the range can be obtained is determined by the field.

Specified by:

`range` in interface `TemporalAccessor`

Parameters:

`field - the field to query the range for, not null`

Returns:

`the range of valid values for the field, not null`

Throws:

`DateTimeException - if the range for the field cannot be obtained`

`UnsupportedTemporalTypeException - if the field is not supported`

get

`public int get(TemporalField field)`

Gets the value of the specified field from this year-month as an `int`.

This queries this year-month for the value of the specified field. The returned value will always be within the valid range of values for the field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return valid values based on this year-month, except PROLEPTIC_MONTH which is too large to fit in an int and throw a DateTimeException. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

get in interface TemporalAccessor

Parameters:

field - the field to get, not null

Returns:

the value for the field

Throws:

DateTimeException - if a value for the field cannot be obtained or the value is outside the range of valid values for the field

UnsupportedTemporalTypeException - if the field is not supported or the range of values exceeds an int

ArithmeticException - if numeric overflow occurs

getLong

```
public long getLong(TemporalField field)
```

Gets the value of the specified field from this year-month as a long.

This queries this year-month for the value of the specified field. If it is not possible to return the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a ChronoField then the query is implemented here. The supported fields will return valid values based on this year-month. All other ChronoField instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoField, then the result of this method is obtained by invoking TemporalField.getFrom(TemporalAccessor) passing this as the

argument. Whether the value can be obtained, and what the value represents, is determined by the field.

Specified by:

`getLong` in interface `TemporalAccessor`

Parameters:

`field` - the field to get, not null

Returns:

the value for the field

Throws:

`DateTimeException` - if a value for the field cannot be obtained

`UnsupportedTemporalTypeException` - if the field is not supported

`ArithmeticException` - if numeric overflow occurs

getYear

```
public int getYear()
```

Gets the year field.

This method returns the primitive `int` value for the year.

The year returned by this method is proleptic as per `get(YEAR)`.

Returns:

the year, from MIN_YEAR to MAX_YEAR

getMonthValue

```
public int getMonthValue()
```

Gets the month-of-year field from 1 to 12.

This method returns the month as an `int` from 1 to 12. Application code is frequently clearer if the enum `Month` is used by calling `getMonth()`.

Returns:

the month-of-year, from 1 to 12

See Also:

`getMonth()`

getMonth

```
public Month getMonth()
```

Gets the month-of-year field using the `Month` enum.

This method returns the enum `Month` for the month. This avoids confusion as to what `int` values mean. If you need access to the primitive `int` value then the enum provides the `int value`.

Returns:

`the month-of-year, not null`

See Also:

`getMonthValue()`

isLeapYear

```
public boolean isLeapYear()
```

Checks if the year is a leap year, according to the ISO proleptic calendar system rules.

This method applies the current rules for leap years across the whole time-line. In general, a year is a leap year if it is divisible by four without remainder. However, years divisible by 100, are not leap years, with the exception of years divisible by 400 which are.

For example, 1904 is a leap year it is divisible by 4. 1900 was not a leap year as it is divisible by 100, however 2000 was a leap year as it is divisible by 400.

The calculation is proleptic - applying the same rules into the far future and far past. This is historically inaccurate, but is correct for the ISO-8601 standard.

Returns:

`true if the year is leap, false otherwise`

isValidDay

```
public boolean isValidDay(int dayOfMonth)
```

Checks if the day-of-month is valid for this year-month.

This method checks whether this year and month and the input day form a valid date.

Parameters:

`dayOfMonth - the day-of-month to validate, from 1 to 31, invalid value returns false`

Returns:

`true if the day is valid for this year-month`

lengthOfMonth

```
public int lengthOfMonth()
```

Returns the length of the month, taking account of the year.

This returns the length of the month in days. For example, a date in January would return 31.

Returns:

`the length of the month in days, from 28 to 31`

lengthOfYear

```
public int lengthOfYear()
```

Returns the length of the year.

This returns the length of the year in days, either 365 or 366.

Returns:

`366 if the year is leap, 365 otherwise`

with

```
public YearMonth with(TemporalAdjuster adjuster)
```

Returns an adjusted copy of this year-month.

This returns a `YearMonth`, based on this one, with the year-month adjusted. The adjustment takes place using the specified adjuster strategy object. Read the documentation of the adjuster to understand what adjustment will be made.

A simple adjuster might simply set the one of the fields, such as the year field. A more complex adjuster might set the year-month to the next month that Halley's comet will pass the Earth.

The result of this method is obtained by invoking the `TemporalAdjuster.adjustInto(Temporal)` method on the specified adjuster passing `this` as the argument.

This instance is immutable and unaffected by this method call.

Specified by:

`with` in interface `Temporal`

Parameters:

`adjuster` - the adjuster to use, not null

Returns:

a YearMonth based on this with the adjustment made, not null

Throws:

`DateTimeException` - if the adjustment cannot be made

`ArithmeticException` - if numeric overflow occurs

with

```
public YearMonth with(TemporalField field,
                      long newValue)
```

Returns a copy of this year-month with the specified field set to a new value.

This returns a `YearMonth`, based on this one, with the value for the specified field changed. This can be used to change any supported field, such as the year or month. If it is not possible to set the value, because the field is not supported or for some other reason, an exception is thrown.

If the field is a `ChronoField` then the adjustment is implemented here. The supported fields behave as follows:

- `MONTH_OF_YEAR` - Returns a `YearMonth` with the specified month-of-year. The year will be unchanged.
- `PROLEPTIC_MONTH` - Returns a `YearMonth` with the specified proleptic-month. This completely replaces the year and month of this object.
- `YEAR_OF_ERA` - Returns a `YearMonth` with the specified year-of-era The month and era will be unchanged.
- `YEAR` - Returns a `YearMonth` with the specified year. The month will be unchanged.
- `ERA` - Returns a `YearMonth` with the specified era. The month and year-of-era will be unchanged.

In all cases, if the new value is outside the valid range of values for the field then a `DateTimeException` will be thrown.

All other `ChronoField` instances will throw an `UnsupportedTemporalTypeException`.

If the field is not a `ChronoField`, then the result of this method is obtained by invoking `TemporalField.adjustInto(Temporal, long)` passing `this` as the

argument. In this case, the field determines whether and how to adjust the instant.

This instance is immutable and unaffected by this method call.

Specified by:

with in interface Temporal

Parameters:

field - the field to set in the result, not null

newValue - the new value of the field in the result

Returns:

a YearMonth based on this with the specified field set, not null

Throws:

DateTimeException - if the field cannot be set

UnsupportedTemporalTypeException - if the field is not supported

ArithmeticException - if numeric overflow occurs

withYear

public YearMonth withYear(int year)

Returns a copy of this YearMonth with the year altered.

This instance is immutable and unaffected by this method call.

Parameters:

year - the year to set in the returned year-month, from MIN_YEAR to MAX_YEAR

Returns:

a YearMonth based on this year-month with the requested year, not null

Throws:

DateTimeException - if the year value is invalid

withMonth

public YearMonth withMonth(int month)

Returns a copy of this YearMonth with the month-of-year altered.

This instance is immutable and unaffected by this method call.

Parameters:

month - the month-of-year to set in the returned year-month, from 1 (January) to 12 (December)

Returns:

a YearMonth based on this year-month with the requested month, not null

Throws:

DateTimeException - if the month-of-year value is invalid

plus

```
public YearMonth plus(TemporalAmount amountToAdd)
```

Returns a copy of this year-month with the specified amount added.

This returns a YearMonth, based on this one, with the specified amount added. The amount is typically Period but may be any other type implementing the TemporalAmount interface.

The calculation is delegated to the amount object by calling TemporalAmount.addTo(Temporal). The amount implementation is free to implement the addition in any way it wishes, however it typically calls back to plus(long, TemporalUnit). Consult the documentation of the amount implementation to determine if it can be successfully added.

This instance is immutable and unaffected by this method call.

Specified by:

plus in interface Temporal

Parameters:

amountToAdd - the amount to add, not null

Returns:

a YearMonth based on this year-month with the addition made, not null

Throws:

DateTimeException - if the addition cannot be made

ArithmeticException - if numeric overflow occurs

plus

```
public YearMonth plus(long amountToAdd,
                      TemporalUnit unit)
```

Returns a copy of this year-month with the specified amount added.

This returns a YearMonth, based on this one, with the amount in terms of the unit added. If it is not possible to add the amount, because the unit is not supported or for some other reason, an exception is thrown.

If the field is a ChronoUnit then the addition is implemented here. The supported fields behave as follows:

- MONTHS - Returns a YearMonth with the specified number of months added. This is equivalent to plusMonths(long).
- YEARS - Returns a YearMonth with the specified number of years added. This is equivalent to plusYears(long).
- DECADES - Returns a YearMonth with the specified number of decades added. This is equivalent to calling plusYears(long) with the amount multiplied by 10.
- CENTURIES - Returns a YearMonth with the specified number of centuries added. This is equivalent to calling plusYears(long) with the amount multiplied by 100.
- MILLENNIA - Returns a YearMonth with the specified number of millennia added. This is equivalent to calling plusYears(long) with the amount multiplied by 1,000.
- ERAS - Returns a YearMonth with the specified number of eras added. Only two eras are supported so the amount must be one, zero or minus one. If the amount is non-zero then the year is changed such that the year-of-era is unchanged.

All other ChronoUnit instances will throw an UnsupportedTemporalTypeException.

If the field is not a ChronoUnit, then the result of this method is obtained by invoking TemporalUnit.addTo(Temporal, long) passing this as the argument. In this case, the unit determines whether and how to perform the addition.

This instance is immutable and unaffected by this method call.

Specified by:

plus in interface Temporal

Parameters:

amountToAdd - the amount of the unit to add to the result, may be negative

unit - the unit of the amount to add, not null

Returns:

a YearMonth based on this year-month with the specified amount added, not null

Throws:

DateTimeException - if the addition cannot be made

UnsupportedTemporalTypeException - if the unit is not supported

ArithmeticException - if numeric overflow occurs

plusYears

public YearMonth plusYears(long yearsToAdd)

Returns a copy of this YearMonth with the specified number of years added.

This instance is immutable and unaffected by this method call.

Parameters:

yearsToAdd - the years to add, may be negative

Returns:

a YearMonth based on this year-month with the years added, not null

Throws:

DateTimeException - if the result exceeds the supported range

plusMonths

public YearMonth plusMonths(long monthsToAdd)

Returns a copy of this YearMonth with the specified number of months added.

This instance is immutable and unaffected by this method call.

Parameters:

monthsToAdd - the months to add, may be negative

Returns:

a YearMonth based on this year-month with the months added, not null

Throws:

DateTimeException - if the result exceeds the supported range

minus

public YearMonth minus(TemporalAmount amountToSubtract)

Returns a copy of this year-month with the specified amount subtracted.

This returns a YearMonth, based on this one, with the specified amount subtracted. The amount is typically Period but may be any other type implementing the TemporalAmount interface.

The calculation is delegated to the amount object by calling `TemporalAmount.subtractFrom(Temporal)`. The amount implementation is free to implement the subtraction in any way it wishes, however it typically calls back to `minus(long, TemporalUnit)`. Consult the documentation of the amount implementation to determine if it can be successfully subtracted.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount to subtract, not null

Returns:

a YearMonth based on this year-month with the subtraction made, not null

Throws:

`DateTimeException` - if the subtraction cannot be made

`ArithmeticException` - if numeric overflow occurs

minus

```
public YearMonth minus(long amountToSubtract,
                       TemporalUnit unit)
```

Returns a copy of this year-month with the specified amount subtracted.

This returns a `YearMonth`, based on this one, with the amount in terms of the unit subtracted. If it is not possible to subtract the amount, because the unit is not supported or for some other reason, an exception is thrown.

This method is equivalent to `plus(long, TemporalUnit)` with the amount negated. See that method for a full description of how addition, and thus subtraction, works.

This instance is immutable and unaffected by this method call.

Specified by:

`minus` in interface `Temporal`

Parameters:

`amountToSubtract` - the amount of the unit to subtract from the result, may be negative

`unit` - the unit of the amount to subtract, not null

Returns:

a YearMonth based on this year-month with the specified amount subtracted, not null

Throws:

DateTimeException - if the subtraction cannot be made

UnsupportedTemporalTypeException - if the unit is not supported

ArithmeticException - if numeric overflow occurs

minusYears

public YearMonth minusYears(long yearsToSubtract)

Returns a copy of this YearMonth with the specified number of years subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

yearsToSubtract - the years to subtract, may be negative

Returns:

a YearMonth based on this year-month with the years subtracted, not null

Throws:

DateTimeException - if the result exceeds the supported range

minusMonths

public YearMonth minusMonths(long monthsToSubtract)

Returns a copy of this YearMonth with the specified number of months subtracted.

This instance is immutable and unaffected by this method call.

Parameters:

monthsToSubtract - the months to subtract, may be negative

Returns:

a YearMonth based on this year-month with the months subtracted, not null

Throws:

DateTimeException - if the result exceeds the supported range

query

public <R> R query(TemporalQuery<R> query)

Queries this year-month using the specified query.

This queries this year-month using the specified query strategy object. The `TemporalQuery` object defines the logic to be used to obtain the result. Read the documentation of the query to understand what the result of this method will be.

The result of this method is obtained by invoking the `TemporalQuery.queryFrom(TemporalAccessor)` method on the specified query passing `this` as the argument.

Specified by:

`query` in interface `TemporalAccessor`

Type Parameters:

`R` - the type of the result

Parameters:

`query` - the query to invoke, not null

Returns:

the query result, null may be returned (defined by the query)

Throws:

`DateTimeException` - if unable to query (defined by the query)

`ArithmeticException` - if numeric overflow occurs (defined by the query)

adjustInto

```
public Temporal adjustInto(Temporal temporal)
```

Adjusts the specified temporal object to have this year-month.

This returns a temporal object of the same observable type as the input with the year and month changed to be the same as this.

The adjustment is equivalent to using `Temporal.with(TemporalField, long)` passing `ChronoField.PROLEPTIC_MONTH` as the field. If the specified temporal object does not use the ISO calendar system then a `DateTimeException` is thrown.

In most cases, it is clearer to reverse the calling pattern by using `Temporal.with(TemporalAdjuster)`:

```
// these two lines are equivalent, but the second approach is recommended
temporal = thisYearMonth.adjustInto(temporal);
temporal = temporal.with(thisYearMonth);
```

This instance is immutable and unaffected by this method call.

Specified by:

`adjustInto` in interface `TemporalAdjuster`

Parameters:

`temporal` - the target object to be adjusted, not null

Returns:

the adjusted object, not null

Throws:

`DateTimeException` - if unable to make the adjustment

`ArithmeticException` - if numeric overflow occurs

until

```
public long until(Temporal endExclusive,
                  TemporalUnit unit)
```

Calculates the amount of time until another year-month in terms of the specified unit.

This calculates the amount of time between two `YearMonth` objects in terms of a single `TemporalUnit`. The start and end points are `this` and the specified year-month. The result will be negative if the end is before the start. The `Temporal` passed to this method is converted to a `YearMonth` using `from(TemporalAccessor)`. For example, the amount in years between two year-months can be calculated using `startYearMonth.until(endYearMonth, YEARS)`.

The calculation returns a whole number, representing the number of complete units between the two year-months. For example, the amount in decades between 2012-06 and 2032-05 will only be one decade as it is one month short of two decades.

There are two equivalent ways of using this method. The first is to invoke this method. The second is to use `TemporalUnit.between(Temporal, Temporal)`:

```
// these two lines are equivalent
amount = start.until(end, MONTHS);
amount = MONTHS.between(start, end);
```

The choice should be made based on which makes the code more readable.

The calculation is implemented in this method for `ChronoUnit`. The units `MONTHS`, `YEARS`, `DECADES`, `CENTURIES`, `MILLENNIA` and `ERAS` are supported. Other `ChronoUnit` values will throw an exception.

If the unit is not a `ChronoUnit`, then the result of this method is obtained by invoking `TemporalUnit.between(Temporal, Temporal)` passing `this` as the first argument and the converted input temporal as the second argument.

This instance is immutable and unaffected by this method call.

Specified by:

`until` in interface `Temporal`

Parameters:

`endExclusive - the end date, exclusive, which is converted to a YearMonth, not null`

`unit - the unit to measure the amount in, not null`

Returns:

`the amount of time between this year-month and the end year-month`

Throws:

`DateTimeException - if the amount cannot be calculated, or the end temporal cannot be converted to a YearMonth`

`UnsupportedTemporalTypeException - if the unit is not supported`

`ArithmeticException - if numeric overflow occurs`

format

```
public String format(DateTimeFormatter formatter)
```

Formats this year-month using the specified formatter.

This year-month will be passed to the formatter to produce a string.

Parameters:

`formatter - the formatter to use, not null`

Returns:

`the formatted year-month string, not null`

Throws:

`DateTimeException - if an error occurs during printing`

atDay

```
public LocalDate atDay(int dayOfMonth)
```

Combines this year-month with a day-of-month to create a `LocalDate`.

This returns a `LocalDate` formed from this year-month and the specified day-of-month.

The day-of-month value must be valid for the year-month.

This method can be used as part of a chain to produce a date:

```
  LocalDate date = year.atMonth(month).atDay(day);
```

Parameters:

`dayOfMonth` - the day-of-month to use, from 1 to 31

Returns:

the date formed from this year-month and the specified day, not null

Throws:

`DateTimeException` - if the day is invalid for the year-month

See Also:

`isValidDay(int)`

atEndOfMonth

```
public LocalDate atEndOfMonth()
```

Returns a `LocalDate` at the end of the month.

This returns a `LocalDate` based on this year-month. The day-of-month is set to the last valid day of the month, taking into account leap years.

This method can be used as part of a chain to produce a date:

```
  LocalDate date = year.atMonth(month).atEndOfMonth();
```

Returns:

the last valid date of this year-month, not null

compareTo

```
public int compareTo(YearMonth other)
```

Compares this year-month to another year-month.

The comparison is based first on the value of the year, then on the value of the month. It is "consistent with equals", as defined by `Comparable`.

Specified by:

`compareTo` in interface `Comparable<YearMonth>`

Parameters:

other - the other year-month to compare to, not null

Returns:

the comparator value, negative if less, positive if greater

isAfter

```
public boolean isAfter(YearMonth other)
```

Checks if this year-month is after the specified year-month.

Parameters:

other - the other year-month to compare to, not null

Returns:

true if this is after the specified year-month

isBefore

```
public boolean isBefore(YearMonth other)
```

Checks if this year-month is before the specified year-month.

Parameters:

other - the other year-month to compare to, not null

Returns:

true if this point is before the specified year-month

equals

```
public boolean equals(Object obj)
```

Checks if this year-month is equal to another year-month.

The comparison is based on the time-line position of the year-months.

Overrides:

equals in class Object

Parameters:

obj - the object to check, null returns false

Returns:

true if this is equal to the other year-month

See Also:

Object.hashCode(), HashMap

hashCode

```
public int hashCode()
```

A hash code for this year-month.

Overrides:

`hashCode` in class `Object`

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

toString

```
public String toString()
```

Outputs this year-month as a `String`, such as `2007-12`.

The output will be in the format `uuuu-MM`:

Overrides:

`toString` in class `Object`

Returns:

a string representation of this year-month, not null

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.